Hidden *Directive*

GF-INC-2026-0704 · DEBRIEF FOR VIRGINIA (GINA) NELAI · SUBMITTED 2026-07-27 · PRIVATE

63 /100 COMPETENT

You ran your queries without process-name filters on purpose, so that guessing the wrong binary could not cost you the finding. That is a deliberate design choice against your own bias, and it is not a common one. It is also why the front of your chain is close to complete while other routes into this incident stalled.

The negatives are handled the same way. You counted 15,123 process events to prove the table was populated before you trusted the NTDS.dit null, so the absence means something. Not-found and ruled-out are kept separate. `m.smith` and `p.singh` are recorded as unresolved rather than concluded in either direction, and both of them are estate noise, so declining to attribute was the right call and you made it without the answer. You state that DCSync-style replication was not checked, rather than letting silence imply it was.

The findings underneath that are strong. Indirect prompt injection identified with `blog_lure.html` and the concealment instruction quoted. The MicrosoftHealthUpdate task tied back to the same `cdn.cloud-endpoint.net/loader` endpoint named in the lure, which is the link that makes the persistence mean something. The `GFUpdaterSvc` hijack with the failed attempt at 12:02:55 and SYSTEM at 12:04:19. The AD Description field and the PowerShell history read at 11:56:40 as the two credential sources, with `cmdkey` correctly held at enumeration only. AdaptixC2 named as the implant. The SRV01 Robocopy alert checked against process telemetry and cleared.

So the shape of what went wrong is worth seeing clearly, because it is narrow.

Three confident conclusions do not hold, and all three are the same move. `WerFault` at 12:26:33 is promoted to a deliberate proxy credential-theft technique when it is Windows Error Reporting responding to the memory access you had already found. `GFDefenderMaintenance` is claimed as a fourth WS01 persistence mechanism when it is estate tooling created by Run Command `script33.ps1` and `script43.ps1`. Defender is claimed to have been disabled by the attacker before the intrusion, and the same artefact was quarantined by Defender at 07:12:52, before your window even opened.

Each of those trusts a shape rather than a source, which is the exact thing your filter discipline exists to prevent. The cost is that every promoted technique becomes work: a client reads a fourth persistence mechanism and goes looking for it, and finds their own tooling. What good looks like is the check you already run on negatives, run on positives too. Before a technique goes in, pull `InitiatingProcessCommandLine` and check whether the same command ran before your window opened. If it did, it is baseline.

The one that costs most is the clear rather than the claim. Collection and exfiltration are actively cleared as false positives, and the `Compress-Archive` at 15:06:52 sits outside the alert window you queried. That is the actual data theft in this incident, and a finding cleared as a false positive is far harder to reopen than a question left open, because nobody revisits a closed door. When you clear something, check that your window covers the event you are clearing, which is the same validation you already apply before trusting a null.

Two absences follow from that. AdminSDHolder is missing entirely, the GenericAll for `svc_backup` at 15:44:59, and DCSync sits at 14:47:51 in `SecurityEvent` 4662. Both outlive every password reset, which is why the impact section reads lighter than the incident was. The anchor at 02:48:26 also sits outside the published window.

You already validate before you believe an absence. Do the same before you believe a clear, and the three overreaches close with it. The instinct that produced the unfiltered queries is the good one, and it is the part that takes longest to learn.

Where the marks went

CATEGORY	SCORE	NOTE
Evidence	17 / 25	Queries deliberately run without process-name filters so a wrong guess could not cause a miss, negatives validated against table coverage before being trusted, and open questions left open. Undercut by three confident conclusions that do not hold.
Kill chain	14 / 25	The front of the chain is close to complete, initial access through lateral movement with correct timestamps, but AdminSDHolder and DCSync are absent and collection and exfiltration are actively cleared as false positives.

CATEGORY	SCORE	NOTE
Scope	9 / 15	Three hosts correct and unresolved accounts honestly marked unresolved, but the incident is anchored at 02:48:26 outside the published window and impact understates because the domain persistence and the data theft are both absent.
Detection	9 / 15	Three detections derived from her own findings, each with a false-positive judgement, and the explorer-then-WerFault pairing is a sharp idea. No KQL, no validation, and one rule rests on a misattributed finding.
Gaps	9 / 10	Separates not-found from ruled-out explicitly, records m.smith and p.singh as unresolved rather than guessing, and states that DCSync-style replication was not checked rather than implying it was.
Craft	8 / 10	The plain-English executive summary is readable end to end by a non-technical reader. No consolidated query appendix, queries are inline only.
Subtotal	66 / 100	
Deduction	-3	Auth event presented as action without object-access or process evidence, the WerFault proxy-read narrative
Total	63 / 100	Competent